2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code) | 979-8-3315-2615-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/LLM4Code66737.2025.00008

♪ With a Little Help from My (LLM) Friends: Enhancing Static Analysis with LLMs to Detect Software Vulnerabilities

Amy Munson*
Computer Science and Engineering
University of California, San Diego

Juanita Gomez,* and Álvaro A. Cárdenas
Computer Science and Engineering
University of California, Santa Cruz

***Abstract*—This paper explores the integration of Large Language Models (LLMs) with static analysis tools, specifically Semgrep, to enhance vulnerability detection in Java applications. Through a series of experiments, we evaluate the performance of various LLMs in triaging security weaknesses identified by Semgrep. We also study how LLMs perform across different types of vulnerabilities and assess the impact of various prompt engineering strategies. Our results reveal that while some LLM models reduce the accuracy of baseline results with static analysis, they show a consistent improvement with each new model released. In particular, o1-mini significantly outperformed others in our experiments in terms of their accuracy and false positive reduction. Although LLMs might not be ready for prime time in vulnerability detection yet, this study highlights their growing potential to complement existing tools and paves the way for future research to further optimize LLM-based vulnerability detection systems.**

***Index Terms*—static analysis, Large Language Models, vulnerability detection**

I. Introduction

Java is one of the most widely used programming languages, powering applications across a broad spectrum, from enterprise software to mobile applications. According to Sonatype's State of the Software Supply Chain report [1], through the first 7 months of 2024, 828 billion Java components were requested from the Maven Central Repository. While Java provides memory safety, various other vulnerabilities may creep into production code and open source repositories, affecting millions of users worldwide. On average, 13 Critical or High-Severity Security vulnerabilities are discovered each year, per application [1]. These numbers underscore the urgent need for better tools to improve security and enhance vulnerability detection.

Static analysis tools such as CodeQL [2] and Semgrep [3] are capable of detecting common security weaknesses in Java code; however, they often produce a large number of false positives and have limited triaging support to help developers address vulnerabilities more effectively. Furthermore, the output formats of these tools are not always easy to interpret, requiring developers to analyze large amounts of information to distinguish true vulnerabilities from noise and extract meaningful security insights.

* These authors contributed equally.

This work studies how Large Language Models (LLMs) can enhance vulnerability triaging by using information from static analysis tools. Our approach combines the strengths of LLMs and static analysis, using LLMs to better categorize vulnerabilities identified by static analyzers. Our experiments show that by using LLMs to process findings from static analysis, we improve triaging accuracy for vulnerabilities such as injection attacks, weak cryptographic algorithms, and path traversal. However, this performance increase is not uniform for all types of vulnerabilities, and for some types, the performance decreases.

With this study, we answer the following research questions:

- **RQ1:** How does integrating LLMs with static analysis tools like Semgrep improve the accuracy, efficiency, and triaging of code vulnerabilities compared to using static analysis tools alone?
- **RQ2:** How effective are different LLMs in enhancing the triaging of code vulnerabilities within a Java codebase, particularly in terms of accuracy, efficiency, and reduction of false positives?
- **RQ3:** How do variations in prompt engineering influence the performance of LLMs in triaging code vulnerabilities, specifically in terms of detection accuracy and false-positive reduction?

Our contributions are as follows.

1) We conduct an empirical study of how different LLMs and prompting techniques perform when detecting a diverse set of Java vulnerabilities by triaging results from a static analysis tool.
2) We test the performance of the latest (at the time of submission) OpenAI o1 "reasoning" models to detect software vulnerabilities.
3) We discuss and provide insights on why some configurations perform better than others for different software vulnerabilities or prompts.

II. Background

A. Vulnerability Detection

Software vulnerability detection refers to the process of identifying weaknesses that could be exploited by attackers. Some key resources for understanding and classifying vulnerabilities include databases like the Common Vulnerabilities

979-8-3315-2615-3/25/$31.00 ©2025 IEEE
DOI 10.1109/LLM4Code66737.2025.00008

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.

and Exposures (CVE) system, which provides a publicly available list of specific known vulnerabilities in software (e.g., a vulnerability in a particular software package), and the Common Weakness Enumeration (CWE) database, which provides general definitions of software weaknesses (e.g., SQL injection).

1) Static Analysis: Static Code Analysis refers to code review to detect potential vulnerabilities in non-executing source code, often through automated tools. Although these tools aim to find security flaws automatically, they primarily help developers by narrowing down security sensitive areas of the code for a more detailed manual review [4].

Static code analysis tools have limitations, particularly with false positives and false negatives. False positives occur when the tool identifies non-existent vulnerabilities due to limited visibility into data flow integrity, particularly when analyzing applications that interact with closed-source components or external systems. False negatives occur when real vulnerabilities go undetected, often because the tool lacks knowledge of new vulnerabilities in external components or details about the security configuration of the running environment [4].

Semgrep is one of the most popular open source static analysis tools designed to identify bugs. Unlike traditional string matching tools, Semgrep enables semantic code searches, allowing pattern matching within code syntax, though it is limited to the analysis of single functions or files [3]. We use Semgrep in our study as a baseline for evaluating the performance of vulnerability detection tools.

2) OWASP Benchmark: The Open Web Application Security Project (OWASP) is a non-profit organization providing resources for securing web applications. One of their projects is the OWASP benchmark, a free Java test suite that evaluates the accuracy, speed, and coverage of automated software vulnerability detection tools.

The benchmark provides a standardized dataset containing both vulnerable and non-vulnerable Java code samples, classified by CWEs, which we use for our study since it allows for reproducibility and objective evaluation of the tools' performance across a variety of security issues derived from real code patterns. We use version v1.2 of the benchmark. There are 2740 total test cases in the benchmark dataset, 1,415 of which contain a true vulnerability and 1,325 do not. Some of the weaknesses in this dataset include:

- **Injection vulnerabilities** (e.g., SQL Injection, Command Injection, LDAP Injection, XPath Injection, and Cross-Site Scripting) exploit improper handling of user inputs, allowing attackers to execute arbitrary commands or manipulate queries.
- **Cryptographic weaknesses** (e.g., Weak Hashing, Weak Cryptography, and Weak Randomness) arise from using insecure algorithms or poorly implemented cryptographic operations.
- **Access and trust boundary issues** (e.g., Path Traversal and Trust Boundary Violations) occur when there is insufficient input validation or improper path construction limitation.
- **Cookie and session vulnerabilities** (e.g., Missing Secure Cookie Flags) involve mishandling session management.

The complete list of code samples divided by CWE can be found in Table I.

TABLE I
OWASP VULNERABLE TESTCASES

CWE	Name	Positive Cases	Negative Cases
22	Path Traversal	133	135
78	Command Injection	126	125
79	XSS	246	209
89	SQL Injection	272	232
90	LDAP Injection	27	32
327	Weak Cryptography	130	116
328	Weak Hashing	129	107
330	Weak Randomness	218	275
501	Trust Boundary	83	43
614	Secure Cookie Flag	36	31
643	XPATH Injection	15	20

Each test case in the benchmark is composed of a Java source code file, e.g. `BenchmarkTest00001.java`, and its corresponding XML file `BenchmarkTest00001.xml` containing metadata and the ground truth. The test cases in the OWASP benchmark have either one type of vulnerability (e.g., CWE 22) or no vulnerability at all. However, non-vulnerable code files include *traps*: code patterns that resemble real vulnerabilities of a specific type but are actually safe.

The XML files contain the benchmark version, vulnerability category, CWE number, test file number, and whether the vulnerability exists. An example of this ground-truth description is illustrated in Figure 1.

```
<test-metadata>
    <benchmark-version>1.2</benchmark-version>
    <category>pathtraver</category>
    <test-number>00001</test-number>
    <vulnerability>true</vulnerability>
    <cwe>22</cwe>
</test-metadata>
```

Fig. 1. XML Labels for the OWASP dataset. This test case shows the file has a CWE 22 vulnerability.

B. Large Language Models

LLMs are deep learning models designed to process and generate human language and pre-trained on vast amounts of data. The foundational structure of many LLMs is the Transformer architecture, which processes input data through multiple layers of "self-attention" to capture context and relevance within text sequences.

The key components of LLMs include tokens, parameters, and temperature. Tokens are text units, like words or subwords, that the model processes to understand and generate responses. The parameters, on the other hand, represent the internal configuration of the model, consisting of weights and biases that influence its decision making. A model with more parameters generally has greater capacity to understand complex

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.

patterns, although this also requires substantial computational resources. The temperature parameter controls the randomness of the model output: Lower values produce more deterministic responses, while higher values increase response diversity and creativity.

For this study, we used five different models: GPT-4, GPT-4o Mini, Claude 3.5 Sonnet, o1-preview, and o1-mini. Each of these has a different base model, maximum number of tokens, and cut-off date indicating when the model's training was last updated, as illustrated in Table II. At the time of the experiments, both o1-preview and o1-mini are beta models. This means that they are previews of unreleased models and do not have all of the features from models such as GPT-4 and GPT-4o Mini.

TABLE II
LLM MODEL DETAILS

Model API	Base Model	Max Tokens	Cut Off
gpt-4-0613	GPT-4	8,192	2021-09
gpt-4o-mini-2024-07-18	GPT-4o Mini	128,000	2023-10
o1-preview-2024-09-12	o1-preview	128,000	2023-10
o1-mini-2024-09-12	o1-mini	128,000	2023-10
claude-3-5-sonnet-20240620	Claude 3.5 Sonnet	200,000	2024-04

1) Prompt Engineering techniques: LLMs rely on an input prompt from a user to elicit a specific language model behavior [5]. Prompt techniques can be as straightforward as directly asking an LLM to complete a task without any examples (Zero Shot), to performing in-context learning by providing examples of the task (e.g., Few Shot) to thought generation (e.g., Chain of Thought) and decomposition (e.g., Tree of Thoughts) techniques to prompt the LLM to articulate its reasoning while solving a problem and decompose the problem into simpler sub-questions.

III. METHODOLOGY

We initially test the OWASP benchmark using Semgrep, and the results are stored in a SARIF file (SARIF is an OASIS standard defining the output file format of static analysis tools).

These SARIF results are imported into a pipeline where we can select an LLM and create prompts for evaluation. Each prompt includes the code from a dataset file and the corresponding SARIF file. The LLM then assesses whether the code contains a specific vulnerability, marking each finding as true or false.

The LLMs are compared to the ground truth in the OWASP dataset and are classified as true positives, false positives, true negatives, or false negatives. To ensure consistency and enable a fair comparison of different LLMs, the same SARIF file is used for all pipeline runs, eliminating variability from rerunning Semgrep.

A. Experiment Prompts

We employ three prompting strategies to evaluate LLMs for vulnerability detection: Zero Shot (ZS), Tree of Thoughts (ToT), and Chain of Thought (CoT). The Zero Shot prompt, based on [6], uses minimal context, relying on the pre-trained capabilities of the LLM. The Tree of Thoughts approach, inspired by [7], decomposes the problem into exploratory paths

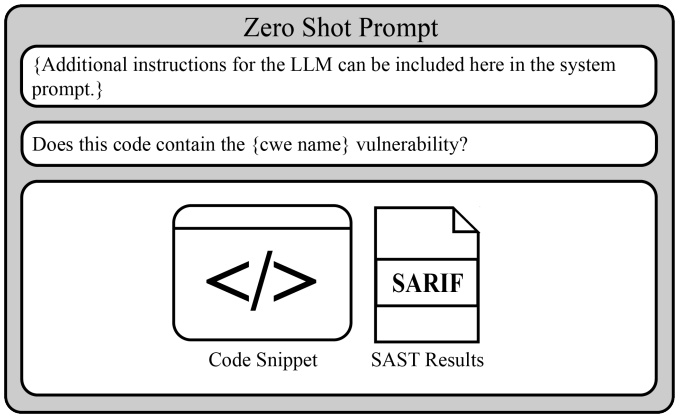


Fig. 2. Zero Shot Prompt Template

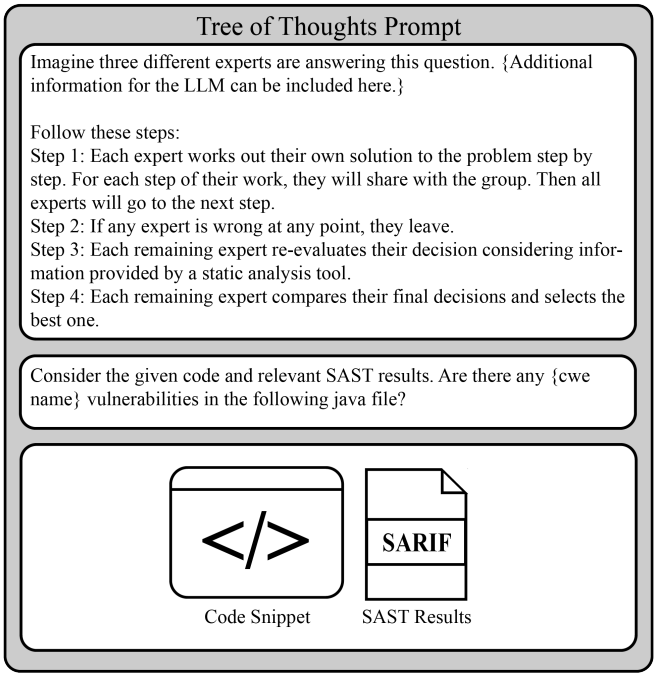


Fig. 3. Tree of Thoughts Prompt Template

to improve reasoning. Finally, the Chain of Thought prompt, guides the LLM through step-by-step reasoning for structured analysis.

The format of these prompts and the additional information are summarized in Figures 2-4. The `cwe name` in the prompt corresponds to the CWE in the ground truth of the test case, as in Figure 1.

B. Evaluation Metrics

To evaluate the performance of LLMs, we use various metrics from classification and machine learning. These metrics provide insights into how accurately the models detect vulnerabilities in the code snippets provided [8]:

- **True Positives (TP)**: Number of correct predictions for the positive class.

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.

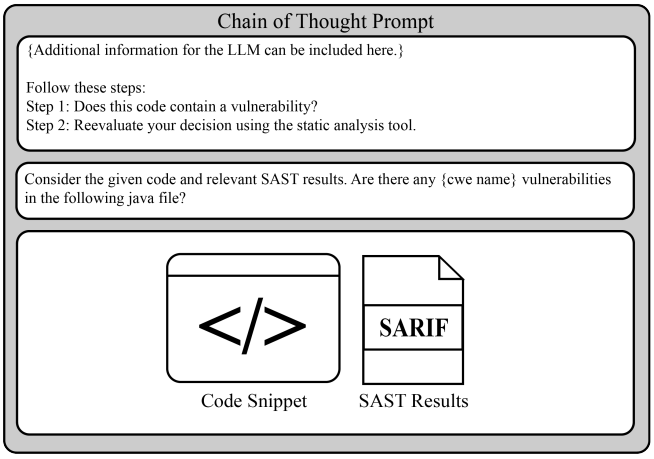


Fig. 4. Chain of Thought Prompt Template

- **False Positives (FP)**: Negative-class instances incorrectly identified as positive cases.
- **False Negatives (FN)**: Positive instances erroneously predicted as negative.
- **True Negatives (TN)**: Negative class instances accurately predicted as negative.

In the context of our study, we define true positives and false positives, using the following notation. Given a file i in the dataset of files $\mathbb{F}$, Let $\mathbb{S}_i$ denote the set of vulnerabilities reported by the static analysis tool in file i. $\mathbb{G}_i$ is the ground truth of the set of actual CWEs in file i (in practice there is at most one CWE associated with each file). We can now formally define the classification metrics as:

$$
\begin{aligned}
&i \text{ is counted as a } TP \text{ iff } \exists s \in \mathbb{S}_i \text{ such that } s \in \mathbb{G}_i,\\
&i \text{ is a } FP \text{ iff } \forall s \in \mathbb{S}_i,\ s \notin \mathbb{G}_i,\\
&i \text{ is a } FN \text{ iff } \forall s \in \mathbb{G}_i,\ s \notin \mathbb{S}_i,\\
&i \text{ is a } TN \text{ iff } \forall s \notin \mathbb{G}_i,\ s \notin \mathbb{S}_i.
\end{aligned}
$$

Out of these core metrics, we compute summary statistics of the performance of the classifier, such as the Accuracy (the proportion of correct predictions over the total number of predictions), the F1 Score (the harmonic mean of Precision and Recall), the ROC Curve (the tradeoff between the True Positive Rate (TPR) and the False Positive Rate (FPR)) and the Area Under the Curve (AUC) (the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative one).

C. Experiments

We evaluate the performance of LLMs in triaging vulnerabilities through three different experiments based on our research questions:

1) ***Comparing Different LLMs****:* In this experiment, we compare the performance of five LLMs previously introduced in analyzing security findings.

To ensure comparability, each model analyzes the complete benchmark dataset using a standardized Chain of Thought prompt shown in Figure 4, with a fixed temperature setting of $T = 0$.

2) ***Comparing LLM and Semgrep Performance****:* The second experiment focuses on a performance comparison between Semgrep with and without the assistance of a LLM. The tools evaluate the OWASP dataset, using the same chain-of-thought prompt as in Experiment 1.

3) ***Comparing Different Prompts****:* In the third experiment, we examine the effects of using different prompts on an LLM's ability to triage results from Semgrep. We tested several types of prompts while maintaining the same temperature $T = 0$, and sample size (200) constant.

IV. Results

1) ***Experiment 1****:* Table III displays the results for the Semgrep baseline and then the five LLMs tested.

The results show that the baseline model, Semgrep, has a true positive rate of 80.49% and a false positive rate of 39.09%. Notably, all LLMs increase the true positive rate while also reducing false positive rates in most cases.

We performed our first experiments in the summer of 2024 before the release of o1-preview and o1-mini by OpenAI in September 2024, and in our initial results, the best-performing model was Claude 3.5 Sonnet. However, as we can see in the table, the new o1-mini significantly increases the performance of vulnerability detection showing higher accuracy, F1-score, and AUC. Although other models have a higher TPR, the reduction in the number of false positives by o1-mini results in a superior overall performance. Given the typically high false positive rates of static analysis tools, reducing false positive rates is one of the most important objectives for triaging.

Claude 3.5 Sonnet ranks second, with high accuracy, F1-score, and AUC, closely followed by o1-preview.

GPT-4o mini achieves a high true positive rate, but also records the highest false positive rate, resulting in the lowest overall accuracy, F1 score, and AUC.

The o1-mini model outperforms o1-preview despite being a smaller model. o1-preview was trained on a more general dataset with broader knowledge, while o1-mini's training was geared toward STEM and coding tasks. This difference might be the reason o1-mini is better in our experiments.

To report the results in Table III, we selected the classifier confidence threshold to maximize the difference between TPR and FPR.

For a more general understanding of the performance of classifiers under different thresholds, we now turn to ROC curves. Figure 5 presents the ROC curves for the five models tested. The ROC curves for o1-mini, o1-preview, and Claude 3.5 Sonnet all consistently sit above GPT-4 and GPT-4o mini due to their superior TPRs and FPRs. Although Claude 3.5 Sonnet and o1-preview have points that sit higher on the graph, o1-mini's curve has a lower false positive rate and a visibly larger AUC as shown in table III.

The graph also reveals a noticeable variation in the number of points in the ROC curves for different models. These points correspond to different TPRs and FPRs as the threshold for the vulnerability probability changes. In particular, o1-preview

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.

TABLE III
LLM PERFORMANCE COMPARISON

Model	TP	FP	TN	FN	TPR	FPR	Accuracy	F1	AUC
Semgrep	1139	518	807	276	80.49%	39.09%	71.02%	74.15%	70.8%
GPT-4	1296	632	693	119	91.59%	47.70%	72.59%	71.20%	79.03%
GPT-4o Mini	1370	787	538	45	96.82%	59.40%	69.64%	66.55%	68.99%
Claude 3.5 Sonnet	1362	412	913	52	96.32%	31.09%	83.06%	82.59%	82.61%
o1-preview	1396	478	847	19	**98.66%**	36.08%	81.86%	81.10%	81.29%
o1-mini	1326	266	1059	89	93.71%	**20.08%**	**87.04%**	**86.92%**	**86.82%**

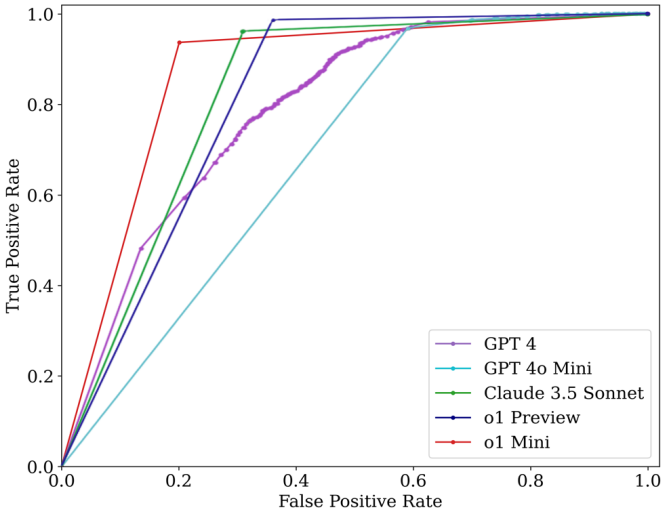


Fig. 5. Different LLMs' ROC Curves for Triaging

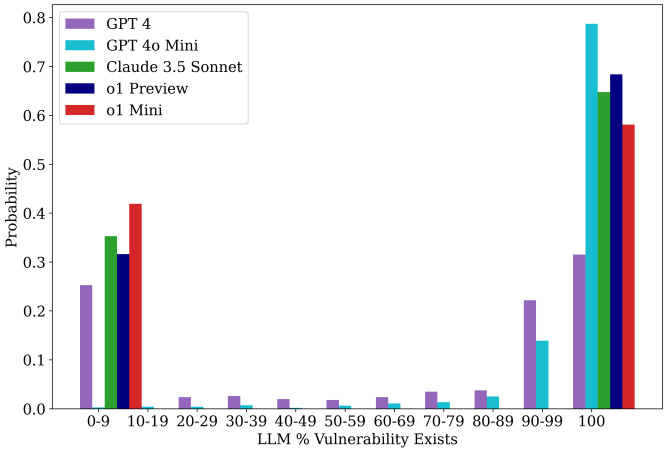


Fig. 6. Probability Mass Function of Percent Probability that a Vulnerability Exists for different LLMs

and o1-mini only have a single point on their ROC curves. This is because they are both beta models without support for features like logprobs, and consequently they do not return any vulnerability probability. Claude 3.5 Sonnet also does not return logprobs like most OpenAI models do, and so it has minimal points as well. The more varied vulnerability probability percentages seen when using GPT-4 result in more visible points in its ROC curve as the threshold for identifying a positive test case is shifted. Figure 6 illustrates the probability of each LLM certainty score for the different models and explains why some LLM models have a more continuous ROC curve trade off than others.

*2) **Experiment 2**:* Now we turn our attention to how Semgrep and LLMs classify individual CWEs to determine whether some CWE vulnerabilities are easier to detect than others. For this we use o1-mini as it performed the best in the previous experiment, and we select GPT-4 as a baseline to illustrate how LLMs are improving over time.

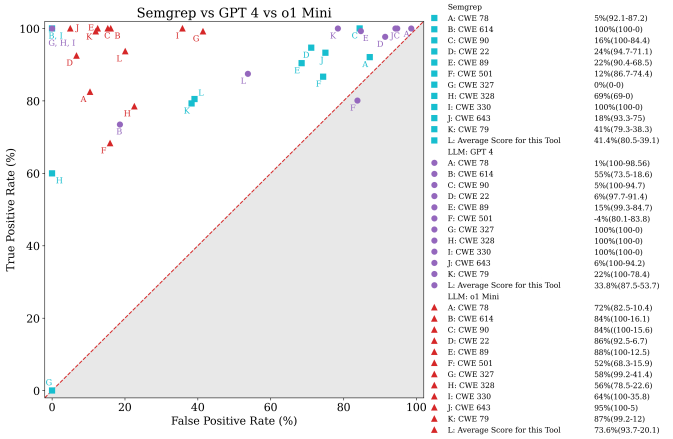


Fig. 7. Performance of Semgrep vs Different LLMs (Using Semgrep)

Figure 7 compares the performance of GPT-4 and o1-mini against Semgrep in identifying individual CWEs. The legend on the right calculates a score for each CWE category by subtracting the FPR from the TPR. The red dotted line represents random guessing, and the shaded area indicates performance levels below random guessing. GPT-4 improves the most in detecting CWEs 327, which correspond to the weak encryption algorithm vulnerability. Although GPT-4 has a true positive rate of 100%, Semgrep does not label any files as containing a weak encryption algorithm within the benchmark.

While GPT-4 performs better in the CWE 327 and 328 categories, it underperforms for the majority of CWEs. In most cases, it detects more true positives than Semgrep, but also reports a higher false positive rate. GPT-4's score for CWE 501 is notable as it falls below both Semgrep and the random guessing threshold.

The o1-mini model, however, performs better than Semgrep in all but three categories (CWE 614, 328, and 330). The o1-

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.

mini average performance in identifying vulnerabilities is also much higher than both GPT-4 and Semgrep.

TABLE IV
COMPARING SEMGREP AND O1-MINI F1-SCORES BY CWE

Rank	CWE	$F1_S$	$F1_o$	$F1_o - F1_S$
1	327	0.0	0.83	0.83
2	643	0.71	0.98	0.27
3	89	0.71	0.94	0.23
4	90	0.717	0.93	0.22
5	22	0.72	0.93	0.21
6	79	0.74	0.94	0.20
7	78	0.67	0.86	0.19
8	501	0.68	0.75	0.07
9	328	0.82	0.79	-0.03
10	614	1.0	0.93	-0.07
11	330	1.0	0.86	-0.14

To summarize these results, we focus on how better (or worse) the o1-mini model performs on individual vulnerabilities when compared to Semgrep.

Table IV, shows a ranked list based on the F1 score difference between the performance of Semgrep ($F1_S$) and the performance of o1-mini ($F1_o$) for each individual vulnerability. We can see how o1-mini has a higher F1-score than Semgrep in most CWE categories. The highest increase in the F1-score from Semgrep to o1-mini is for CWE 327 (weak cryptography). In this case, Semgrep failed to identify most of the weak cryptography cases, so this is an example of the LLM overriding the assessment of the SARIF file and indicating that there is indeed a vulnerability, even if the SARIF file states that there is none.

There are three vulnerabilities where o1-mini performs worse than Semgrep: CWE 328, 614, and 330. The significantly higher false positive rate for all three of these categories when evaluated by o1-mini significantly impacts these scores even though the true positive rate remains the same or improves. This highlights an area for improvement in LLM-based detection, particularly for weak randomness and weak hashing vulnerabilities.

o1-mini's improved F1-scores for CWEs such as CWE 22, 79, and 89 in contrast to the lower scores for CWEs 501, 328, and 327 indicate that the LLM is better able to identify vulnerabilities that do not require additional context.

Figure 8 shows the three most significant improvements of o1-mini over Semgrep (from table IV). We can see that for CWE 327 (weak cryptography), the LLM increases the detection rate at the cost of some false positives. However, for CWE 89 (SQL injection) and CWE 643 (XPATH injection), the LLM significantly reduces the false positive rate, while slightly increasing the true positive rates.

Figure 9 similarly shows the three cases in which o1-mini underperformed compared to Semgrep. Although the detection rate either improved or remained the same, the increased false positive rate negatively affected the overall F1 score. In a high-criticality setting, where missing vulnerabilities poses a greater risk than handling false positives, this trade-off might be acceptable. Therefore, we argue that o1-mini generally outperforms Semgrep if minimizing false negatives is prioritized over reducing false alerts.

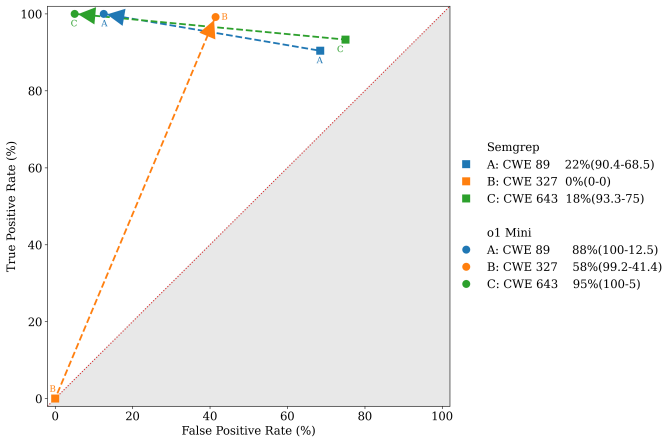


Fig. 8. Semgrep vs o1-mini Performance for Best CWEs by F1-Score Improvement

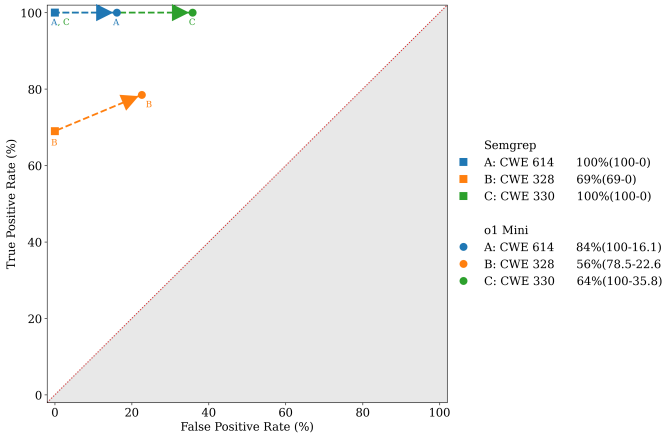


Fig. 9. Semgrep vs o1-mini Performance for Worst CWEs by F1-Score Improvement

*3) **Experiment 3**:* We now discuss the performance of different prompts. Figure 10 illustrates the performance of different prompts with the same LLM models tested in Experiment 2.

For GPT-4 we can clearly see that the Tree of Thoughts prompt curve clearly lies above all other GPT-4 curves, showing that more efforts in prompt engineering may increase the performance of the models.

Notice, however, that despite being the best performing prompt for GPT-4, the Tree of Thoughts prompt curve for GPT-4 is still in the convex hull of the worst performing o1-mini prompt (Zero Shot).

Table V shows more detailed results for the o1-mini case. We can see that the o1-mini model, the more complex prompts performed similarly to even the simple Zero Shot prompt. While a more complicated Chain of Thought or Tree of Thoughts prompt was still a noticeable improvement over the Zero Shot prompt (particularly in regard to the FPR), the difference in their accuracy, F1-score, and AUC were all under 0.3%.

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.

TABLE V
LLM AND PROMPT PERFORMANCE COMPARISON

Model	Prompt	TP	FP	TN	FN	TPR	FPR	Accuracy	F1	AUC
o1-mini	Zero Shot	1360	375	950	55	**96.11%**	28.30%	84.31%	83.95%	83.91%
o1-mini	Chain of Thought	1326	266	1059	89	93.71%	20.08%	**87.04%**	**86.92%**	**86.82%**
o1-mini	Tree of Thoughts	1320	263	1062	95	93.29%	**19.85%**	86.93%	86.82%	86.72%

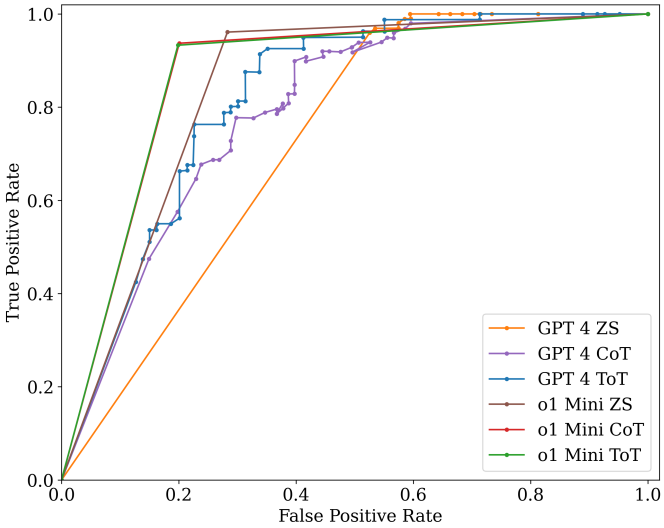


Fig. 10. ROC Curves for Different Prompt Styles and LLMs

A. Summary and Analysis

Our study demonstrates the potential of combining LLMs with static analysis tools such as Semgrep for vulnerability detection. The results indicate that while GPT-4, which was state-of-the-art a couple of months ago and proved to be impressive on a variety of tasks, fell short on vulnerability detection. However, newer "reasoning" models, such as o1-mini, surpass the performance of previous models, achieving significantly better accuracy and lower false positive rates. For example, o1-mini exhibited the highest accuracy (87.04%) and F1 score, suggesting that newer models are progressing toward achieving reliable vulnerability detection and triaging.

We also show that the performance of LLMs is significantly dependent on the type of vulnerability. Weak cryptography seems to be a strong suit for LLMs, while finding weak randomness seems to be hard for them. In future work, we will attempt to find the reasons for this behavior.

Our findings also show that, depending on the model, prompt engineering can play a critical role in using LLMs full capabilities. Models like GPT-4 benefit greatly from better prompt design, while newer models like o1-mini require less prompt engineering effort since they were designed to perform complex *reasoning* by "thinking" before answering, so while a CoT or ToT prompt might improve their performance, the gains are not as significant when applying these prompts to earlier models.

a) Limitations: One of the limitations of our study is that the evaluation setup is highly specific, relying on a fixed benchmark and a limited set of CWEs, which may not fully represent real-world vulnerability distributions. Our study assumes that each vulnerability corresponds to a single CWE, whereas, in practice, vulnerabilities often exhibit multiple CWE classifications. This suggests that the problem may be more appropriately framed as a multi-label classification task, requiring different evaluation metrics. Finally, another potential problem is data contamination, as the models may have been trained on the OWASP benchmark; this raises the need to regularly update benchmarks to test newer LLM models with tests that could not have been seen by the LLMs before.

V. RELATED WORK

LLMs have been explored for a range of complex tasks within software engineering. Wu et al. [9] highlights LLMs strengths in code generation and reasoning tasks, and Feng and Chen [10] use LLMs to automate bug reproduction, outperforming existing methods in speed and accuracy. Yuan et al. [11] show that fine-tuning LLMs improves their code comprehension and generation abilities, while Zhang et al. [12] apply LLMs to resolve GitHub issues, achieving better results than traditional tools.

In terms of software security, LLMs have been used in vulnerability detection. Ullah et al. [6] and Ding et al. [13] highlight the limitations of LLMs in reliably identifying security vulnerabilities. In particular, LLMs such as GPT-4 and PaLM2 struggle with accurately detecting bugs [6]. Other approaches like improved labeling [13], customized prompts [14], and retrieval-augmented generation [15] have been attempted but still show that current LLMs appear to be unreliable for such tasks.

Other works study the use of LLMs for vulnerability repair. Pearce et al. [16] explore the use of LLMs for zero-shot vulnerability repair, highlighting both the potential and challenges in generating functionally correct code while addressing security bugs. Islam et al. [17] introduce SecRepair, which combines reinforcement learning with LLMs to generate secure code fixes and vulnerability descriptions. Similarly, Fitero-Dominguez et al. [18] present a fine-tuning approach with LLMs to improve accuracy in vulnerability repair. Although these methods show potential, they still face challenges in producing accurate, secure patches, often requiring further human refinement and contextual understanding.

Despite these challenges, LLMs can be effective in detecting certain types of vulnerabilities with advanced prompting techniques, pointing to the potential of LLMs when combined with neuro-symbolic approaches [19], [20]. Symbolic tools, such as static analyzers, have the ability to assist in code development. Previous work has focused on comparing static analysis tools for vulnerability detection [21], [22] and

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.

web application security testing [23], but comparatively less emphasis has been placed on enhancing the capabilities of static analysis tools through artificial intelligence for this particular purpose. LLift [24] combines static analysis with LLMs for bug detection in the Linux kernel, demonstrating the potential of AI to enhance traditional analysis, and IRIS [25] uses a neuro-symbolic approach that integrates LLMs with CodeQL to infer taint specifications. Although promising, these approaches are domain-specific, focusing on particular systems or vulnerability classes.

Our work addresses this gap by combining LLMs with Semgrep for a more comprehensive investigation on how LLMs can improve the performance of static analysis tools with different prompt techniques, models, and vulnerability classes.

VI. Conclusions and Future Work

This study shows promising progress towards the practical use of LLMs to improve software vulnerability detection. LLMs have the potential to reduce false positives, accelerate the triaging processes, and eventually eliminate the long and arduous task of triaging vulnerabilities in software.

In future work, we plan to evaluate LLM-based vulnerability detection on more complex and realistic datasets. The OWASP Benchmark consists of Java code examples, each one corresponding to a single CWE and are contained within a single file. In practice, real-world vulnerabilities often span multiple files, involve multiple CWEs, and appear in various programming languages. Expanding our evaluation to include datasets that reflect these complexities will provide deeper insight into the strengths and weaknesses of LLMs in enhancing static analysis tools. Furthermore, since our findings suggest that LLMs introduce a trade-off between improved detection and higher false-positive rates, future work should explore hybrid approaches that combine LLM reasoning with rule-based static analysis to balance precision and recall in security analysis.

VII. Acknowledgments

We acknowledge AppSecAI Inc. for providing the platform and support essential to conduct the high volume of experiments that underpin this paper. Research was sponsored by the Army Research Office and was accomplished under Grant Number W911NF-23-1-0373. For this research, we also collaborated with OpenAI through the Cybergrant program, using their API credits and funding to advance our research in AI-driven cybersecurity solutions.

References

[1] Sonatype. State of the software supply chain. [Online]. Available: https://www.sonatype.com/state-of-the-software-supply-chain/2024/thank-you?submissionGuid=6a563d29-a42c-4f9e-b9ed-42ce46109a9d

[2] CodeQL. Codeql. [Online]. Available: https://codeql.github.com

[3] I. Semgrep. Semgrep. [Online]. Available: https://github.com/semgrep/semgrep

[4] O. Foundation. (2024) Owasp static code analysis. [Online]. Available: https://owasp.org/www-community/controls/Static_Code_Analysis

[5] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff *et al.*, "The prompt report: A systematic survey of prompting techniques," *arXiv preprint arXiv:2406.06608*, 2024.

[6] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," 2024. [Online]. Available: https://arxiv.org/abs/2312.12575

[7] D. Hulbert, "Using tree-of-thought prompting to boost chatgpt's reasoning," https://github.com/dave1010/tree-of-thought-prompting, May 2023. [Online]. Available: https://doi.org/10.5281/zenodo.10323452

[8] IBM. (2024) What is a confusion matrix? [Online]. Available: https://www.ibm.com/topics/confusion-matrix

[9] F. Wu, Q. Zhang, A. P. Bajaj, T. Bao, N. Zhang, R. F. Wang, and C. Xiao, "Exploring the limits of chatgpt in software security applications," 2023. [Online]. Available: https://arxiv.org/abs/2312.05275

[10] S. Feng and C. Chen, "Prompting is all you need: Automated android bug replay with large language models," 2024. [Online]. Available: https://arxiv.org/abs/2306.01987

[11] Z. Yuan, J. Liu, Q. Zi, M. Liu, X. Peng, and Y. Lou, "Evaluating instruction-tuned large language models on code comprehension and generation," 2023. [Online]. Available: https://arxiv.org/abs/2308.01240

[12] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," 2024. [Online]. Available: https://arxiv.org/abs/2404.05427

[13] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" 2024. [Online]. Available: https://arxiv.org/abs/2403.18624

[14] B. Steenhoek, M. M. Rahman, M. K. Roy, M. S. Alam, E. T. Barr, and W. Le, "A comprehensive study of the capabilities of large language models for vulnerability detection," 2024. [Online]. Available: https://arxiv.org/abs/2403.17218

[15] X. Du, G. Zheng, K. Wang, J. Feng, W. Deng, M. Liu, B. Chen, X. Peng, T. Ma, and Y. Lou, "Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag," 2024. [Online]. Available: https://arxiv.org/abs/2406.11147

[16] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," 2022. [Online]. Available: https://arxiv.org/abs/2112.02125

[17] N. T. Islam, J. Khoury, A. Seong, M. B. Karkevandi, G. D. L. T. Parra, E. Bou-Harb, and P. Najafirad, "Llm-powered code vulnerability repair with reinforcement learning and semantic reward," 2024. [Online]. Available: https://arxiv.org/abs/2401.03374

[18] D. de Fitero-Dominguez, E. Garcia-Lopez, A. Garcia-Cabot, and J.-J. Martinez-Herraiz, "Enhanced automated code vulnerability repair using large language models," *Engineering Applications of Artificial Intelligence*, vol. 138, p. 109291, Dec. 2024. [Online]. Available: http://dx.doi.org/10.1016/j.engappai.2024.109291

[19] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2311.16169

[20] Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai, "Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2402.17230

[21] G. Bennett, T. Hall, E. Winter, and S. Counsell, "Semgrep*: Improving the limited performance of static application security testing (sast) tools," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 614–623. [Online]. Available: https://doi.org/10.1145/3661167.3661262

[22] L. Kree, R. Helmke, and E. Winter, "Using semgrep oss to find owasp top 10 weaknesses in php applications: A case study," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, F. Maggi, M. Egele, M. Payer, and M. Carminati, Eds. Cham: Springer Nature Switzerland, 2024, pp. 64–83.

[23] K. Kuszczyński and M. Walkowski, "Comparative analysis of open-source tools for conducting static code analysis," *Sensors*, vol. 23, no. 18, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/18/7978

[24] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An llm-integrated approach," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA1, Apr. 2024. [Online]. Available: https://doi.org/10.1145/3649828

[25] Z. Li, S. Dutta, and M. Naik, "Llm-assisted static analysis for detecting security vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2405.17238

Authorized licensed use limited to: Mysore College of Engineering and Management. Downloaded on July 09,2026 at 09:22:02 UTC from IEEE Xplore. Restrictions apply.